

# Aegis — Industry-Grade LLM Jailbreak & Prompt-Injection Defense

## Technical Design & Build Roadmap (v1.0)

Successor to RJD-v2. Working name: **Aegis** (the layered guardrail system); **RJD-3** is its core detection model. Rename freely.

**Goals (all three):** a publishable **research result**, a polished **open-source model + library**, and a deployable **production service** — built on **free Kaggle/Colab T4** compute, covering the **full layered system**.

---

### 0. Honest framing (read this first)

Three truths shape every decision below:

- No single defense is unbreakable.** The 2026 consensus — “*The Attacker Moves Second*” — is that adaptive attackers bypass most individual defenses, including fine-tuned ones. The goal is **defense-in-depth that raises attacker cost**, not a silver bullet.
- “Best” is a moving target.** Guard models and attacks refresh every few months (Llama Guard 4, Qwen3Guard, ShieldGemma 2 all landed in 2025). A credible system needs **continuous red-teaming and retraining**, not a one-shot train.
- T4 reality.** We cannot train a frontier guard from scratch. We **LoRA-fine-tune small guards (1–4B)**, **ensemble open guards**, and win on **architecture + data + evaluation rigor** — which is exactly where the academic frontier is.

What we *can* credibly achieve: a layered system that matches or beats single open guards on standard benchmarks, with a reproducible paper, an installable library, and a deployable service.

---

### 1. Threat model (2026)

Mapped to **OWASP LLM01 (Prompt Injection)** — still the #1 LLM risk — and adjacent categories.

| Threat                      | Description                                                                                                 | Where it hits |
|-----------------------------|-------------------------------------------------------------------------------------------------------------|---------------|
| <b>Direct jailbreak</b>     | DAN, roleplay, refusal-suppression, obfuscation (Base64, homoglyph, zero-width, emoji-smuggling, bidi text) | Chat input    |
| <b>Multi-turn jailbreak</b> | Attack split across turns to                                                                                | Chat session  |

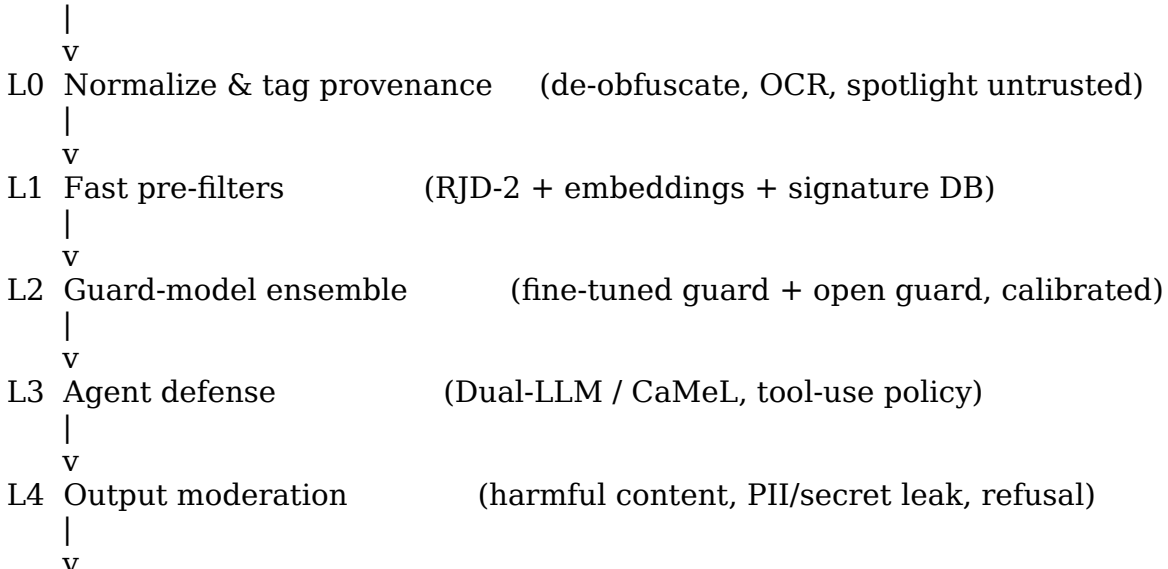
|                             |                                                                   |               |
|-----------------------------|-------------------------------------------------------------------|---------------|
|                             | evade single-turn classifiers                                     |               |
| <b>Indirect injection</b>   | Malicious instructions hidden in retrieved web/email/docs         | RAG, agents   |
| <b>Tool / MCP poisoning</b> | Hostile instructions in tool descriptions or tool outputs         | Agents, MCP   |
| <b>Multimodal injection</b> | Payloads in images, QR codes, steganography                       | Vision models |
| <b>Adaptive attacks</b>     | Optimizer- or LLM-generated attacks tuned to <i>our</i> defense   | All layers    |
| <b>Output-side</b>          | Data/secret exfiltration, harmful generation that slipped through | Responses     |

## 2. Design principles

- **Layered & independent** — bypassing one layer must not bypass the system.
- **Provenance-first** — every token is tagged trusted vs. untrusted (spotlighting); untrusted data can never be treated as instructions.
- **Least privilege for agents** — tools that write/transfer/escalate are gated when a turn consumed untrusted content.
- **Measured, not asserted** — every layer has a benchmark and an adaptive-attack test.
- **Calibrated & observable** — trustworthy scores, full logging, drift alarms.
- **Continuous** — automated red-teaming and retraining are first-class, not afterthoughts.

## 3. Reference architecture

Untrusted input (prompt / retrieved content / tool output)



allow / escalate / block

L5 Continuous ops (red-team - monitor - drift - retrain) — wraps every layer

| Layer                            | What it does                                                                                                                                                                                | T4-feasible build                                      |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>L0 Normalize + provenance</b> | NFKC, homoglyph/zero-width/emoji-smuggling removal, Base64/ROT13 decode, OCR for images, <b>spotlight</b> untrusted spans with delimiters/encoding                                          | Extend RJD-v2 normalize.py; add pytesseract/Pillow OCR |
| <b>L1 Fast pre-filters</b>       | Cheap CPU triage: RJD-v2 classifier + <b>embedding similarity</b> to a <b>known-attack DB</b> + regex heuristics                                                                            | RJD-v2 + bge-small/gte-small embeddings + FAISS        |
| <b>L2 Guard ensemble</b>         | The “model”: a <b>LoRA-fine-tuned small guard</b> (Qwen3-1.7/4B or Llama-3.2-3B) <b>ensembled</b> with an open guard (Qwen3Guard / Llama Guard 4) using ANY-flag for high-stakes categories | LoRA on T4; open guards via HF/Transformers            |
| <b>L3 Agent defense</b>          | <b>Dual-LLM</b> (Privileged planner + Quarantined data-processor) / <b>CaMeL</b> capability tracking; tool-use policy; human confirm for high-risk actions                                  | Orchestrate small/open models; evaluate on AgentDojo   |
| <b>L4 Output moderation</b>      | Response safety classifier + <b>secret/PII leak</b> detection + refusal-consistency                                                                                                         | Guard model in response mode + detectors               |
| <b>L5 Continuous ops</b>         | Automated red-teaming (garak, PyRIT), drift & anomaly monitoring, feedback retraining                                                                                                       | Scheduled jobs; eval gates                             |

## 4. Data strategy

A fresh, de-duplicated, contamination-controlled corpus is the single biggest lever.

- **Public sources:** TrustAIRLab in-the-wild jailbreaks; **HarmBench**, **JailbreakBench** (JBB-Behaviors), **AdvBench**; **WildGuardMix** / **WildJailbreak** (AI2); **AgentDojo** &

**InjecAgent** (indirect/agent); **OS-Harm** (computer-use); ToxicChat, BeaverTails; benign hard-negatives (sensitive-but-OK prompts) to control over-refusal.

- **Adversarial augmentation:** our attack suite (paraphrase, leetspeak, homoglyph, Base64, ROT13, zero-width, payload-split, emoji-smuggling, bidi) applied to positives.
- **Synthetic generation:** use an LLM to generate fresh jailbreak variants and indirect-injection documents (held out from training for an honest “unseen” split).
- **Hygiene:** strict train/val/test isolation, benchmark prompts kept **test-only** to avoid contamination, balanced and imbalanced eval views.

## 5. Modeling plan (T4-scoped)

1. **Core classifier (L1):** keep/upgrade RJD-v2 (de-obfuscation + TF-IDF + features + calibrated ensemble) as the millisecond pre-filter.
2. **Semantic layer (L1):** sentence-embedding nearest-neighbor to a curated known-attack DB → catches paraphrase/novel-but-similar.
3. **Fine-tuned guard (L2): LoRA / QLoRA** fine-tune a small instruct model (Qwen3-1.7B or 4B) on the corpus in both **prompt-classification** and **response-classification** modes; optionally **SecAlign-style preference optimization** to harden instruction/data separation.
4. **Ensemble + calibration:** soft-vote/stack the fine-tuned guard with an open guard; **Platt/temperature** calibration; per-category thresholds; selective **cascade** (cheap layers decide the easy 80–90%, escalate the rest).
5. **Distillation (optional):** distill the ensemble into one small student for cheap serving.

## 6. Evaluation protocol (the part that makes it credible)

- **Benchmarks:** HarmBench, JailbreakBench, AdvBench (jailbreaks); **AgentDojo**, **InjecAgent**, **OS-Harm**, **LLMail-Inject** (agent/indirect); our own held-out unseen-attack set.
- **Metrics:** Attack Success Rate (!), recall @ low FPR, robust recall per attack, **over-refusal (FRR)** on hard benign, ROC-AUC, ECE (calibration), latency, agent task-utility-under-attack.
- **Adaptive evaluation (mandatory):** run optimizer/LLM attacks *against Aegis itself* (the “attacker moves second” test). Report robustness to adaptive attacks, not just static ones.
- **Red-team tooling:** garak, Microsoft PyRIT; track results over time.
- **Baselines:** Llama Guard 4, Qwen3Guard, ShieldGemma 2, WildGuard, plus our RJD-v2 — same prompts, same protocol.

## 7. The three deliverable tracks (built from one codebase)

| Track           | Deliverable                                                                       | Notes                                                      |
|-----------------|-----------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Research</b> | Paper: layered design + adaptive-attack eval + ablations + results tables/figures | Targets a workshop/arXiv; honest negative results included |

|                    |                                                                                                                                                  |                                                 |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <b>Open-source</b> | aegis Python library<br>(aegis.scan(text),<br>aegis.guard_agent(...)),<br>the RJD-3 guard on<br>HuggingFace, model card,<br>docs                 | Apache/MIT; reproducible<br>notebook            |
| <b>Production</b>  | FastAPI service: layered<br>pipeline, caching, per-<br>tenant policy,<br>OpenTelemetry monitoring,<br>OWASP/NIST control<br>mapping, latency SLA | vLLM for the guard model;<br>Docker; load tests |

## 8. Phased roadmap (each phase ships something runnable on T4)

| Phase                   | Focus                                                                                                  | Key output                                                                                     |
|-------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>P0 — Setup</b>       | Repo scaffold, env, data<br>licenses, eval harness<br>skeleton                                         | aegis/ repo + CI +<br>benchmark loaders                                                        |
| <b>P1 — Data + Eval</b> | Assemble & clean the<br>corpus; wire up<br>HarmBench/JailbreakBench<br>/AgentDojo loaders +<br>metrics | Versioned dataset + one-<br>command eval harness +<br>baseline numbers (RJD-2,<br>open guards) |
| <b>P2 — L0 + L1</b>     | Upgrade normalization<br>(emoji/bidi/OCR/spotlight)<br>+ embedding/signature<br>pre-filters            | Fast layer beating RJD-v2 on<br>obfuscation + novel-similar                                    |
| <b>P3 — L2 guard</b>    | LoRA-fine-tune small guard;<br>ensemble + calibrate;<br>(optional SecAlign)                            | RJD-3 guard model on HF +<br>ensemble results vs<br>baselines                                  |
| <b>P4 — L3 agent</b>    | Dual-LLM/CaMeL + tool-use<br>policy; evaluate on<br>AgentDojo/InjecAgent                               | Agent-defense module +<br>AgentDojo scores                                                     |
| <b>P5 — L4 + L5</b>     | Output moderation + leak<br>detection; red-team loop +<br>monitoring + retrain                         | Full pipeline + adaptive-<br>attack report                                                     |
| <b>P6 — Package</b>     | Library release, paper<br>write-up, FastAPI service +<br>Docker, docs                                  | Paper draft + pip install<br>aegis + deployable API                                            |

*Suggested first move: **P0 + P1** — stand up the evaluation harness and baselines. Everything else is judged against it, so it comes first.*

## 9. Risks, limitations & ethics

- **Adaptive attacks will find gaps** — we report this honestly and measure cost-to-bypass, not “unbreakable.”
- **Over-refusal** is the silent failure mode — hard-benign evaluation is mandatory or we ship an unusable filter.
- **Maintenance burden** — without continuous red-teaming the system decays; the roadmap bakes this in (P5).
- **Dual-use / responsible disclosure** — attack code stays defensive; we don’t publish turnkey weaponized exploits; align with OWASP/NIST and responsible-disclosure norms.
- **Compute honesty** — on T4 we won’t out-train Meta/Google; our edge is architecture, data freshness, ensembling, and evaluation rigor.

## 10. Tech stack & repo layout

|                |                                                 |
|----------------|-------------------------------------------------|
| aegis/         |                                                 |
| └─ aegis/      |                                                 |
| └─ normalize/  | L0 — de-obfuscation, OCR, spotlighting          |
| └─ prefilter/  | L1 — RJD-2 classifier, embeddings, signature DB |
| └─ guard/      | L2 — fine-tuned guard + ensemble + calibration  |
| └─ agent/      | L3 — dual-LLM / CaMeL, tool-use policy          |
| └─ output/     | L4 — response moderation, leak detection        |
| └─ ops/        | L5 — red-team runners, monitors, retrain jobs   |
| └─ pipeline.py | orchestration + cascade                         |
| └─ eval/       | benchmark loaders, metrics, adaptive attacks    |
| └─ data/       | assembly scripts + manifests (no raw dumps)     |
| └─ service/    | FastAPI app, Dockerfile, OTel                   |
| └─ notebooks/  | Kaggle/Colab T4 training & eval                 |
| └─ docs/       | model cards, OWASP/NIST mapping, paper          |

**Stack:** Python, scikit-learn, PyTorch, Transformers + PEFT/LoRA, sentence-transformers + FAISS, vLLM (serving), FastAPI, garak + PyRIT (red-team), Weights & Biases (tracking), HuggingFace Hub (release).

## 11. References (2026 state of the art)

- OWASP Top 10 for LLM Applications 2025 — LLM01 Prompt Injection (genai.owasp.org)
- Llama Guard 4 model card (llama.com); Qwen3Guard Technical Report (arXiv:2510.14276); benchmarking guard models, ICLR 2026 (arXiv:2605.28830)
- CaMeL / Design Patterns for Securing LLM Agents (arXiv:2506.08837); SecAlign + StruQ (BAIR, 2025); Spotlighting (arXiv:2403.14720)
- “The Attacker Moves Second” — adaptive attacks bypass defenses (arXiv:2510.09023)
- AgentDojo (indirect injection for agents); InjecAgent; OS-Harm (arXiv:2506.14866); LLMail-Inject (Microsoft)
- NIST AI Risk Management Framework (Govern/Map/Measure/Manage)
- Commercial bar: Lakera Guard (Check Point), Cisco AI Defense (Robust Intelligence), Prisma AIRS (Palo Alto/Protect AI), HiddenLayer, Mindgard